

Machine Learning Lab Programs

Name: Avyukt Gupta

Reg no : 2427010161

Experiments and Datasets:

1. Iris — Data Loading, Train-Test Split & Feature Engineering
2. California Housing — Linear Regression
3. Breast Cancer Wisconsin — Logistic Regression with Regularization
4. Wine — KNN vs Decision Tree
5. Iris — SVM with Different Kernels
6. Digits + manual AND-gate data — Perceptron & Neural Network

Experiment 1: Data Loading, Train-Test Split & Feature Engineering

Aim: To load the Iris dataset, split it into training and testing sets, and standardize its features.

Code:

Dataset used: *Iris dataset (sklearn.datasets.load_iris)*

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

iris = load_iris()
X = pd.DataFrame(iris.data, columns=iris.feature_names)
y = pd.Series(iris.target, name="species")
print("Dataset shape:", X.shape)
print(X.head())
print("\nClass distribution:\n", y.value_counts())
print("\nMissing values:\n", X.isnull().sum())

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=.2, random_state=42, stratify=y)
print("\nTraining set size:", X_train.shape)
print("Testing set size:", X_test.shape)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
print("\nBefore scaling (first row):", X_train.iloc[0].values)
```

```
print("After scaling (first row): ", X_train_scaled[0])

fig, ax = plt.subplots(1, 2, figsize=(12, 4))
ax[0].boxplot(X_train.values); ax[0].set_title("Before Scaling")
ax[0].set_xticklabels(X.columns, rotation=45)
ax[1].boxplot(X_train_scaled); ax[1].set_title("After Scaling")
ax[1].set_xticklabels(X.columns, rotation=45)
```

Output:

```
Dataset shape: (150, 4)
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)
0                5.1                3.5                1.4                0.2
1                4.9                3.0                1.4                0.2
2                4.7                3.2                1.3                0.2
3                4.6                3.1                1.5                0.2
4                5.0                3.6                1.4                0.2

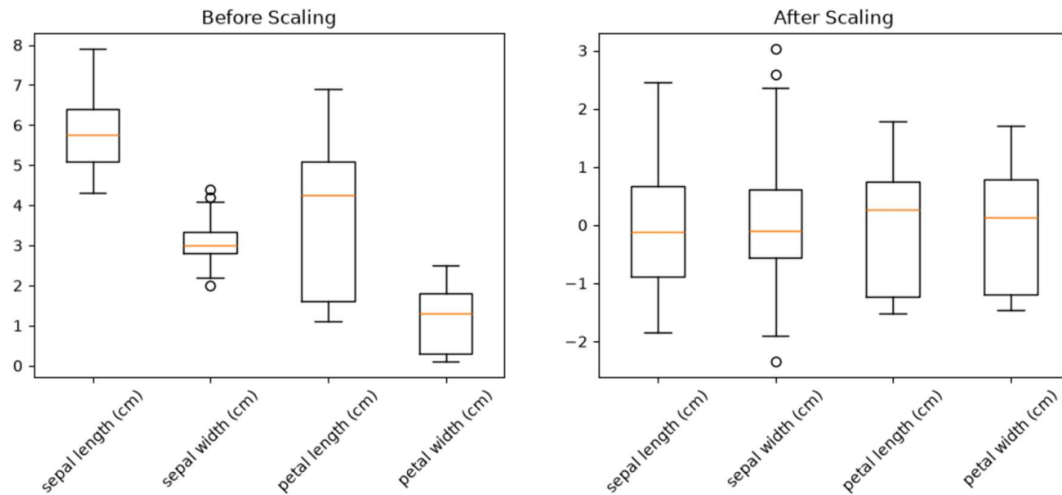
Class distribution:
   species
0       50
1       50
2       50
Name: count, dtype: int64

Missing values:
   sepal length (cm)    0
   sepal width (cm)    0
   petal length (cm)   0
   petal width (cm)    0
dtype: int64

Training set size: (120, 4)
Testing set size: (30, 4)

Before scaling (first row): [4.4 2.9 1.4 0.2]
After scaling (first row): [-1.72156775 -0.33210111 -1.34572231 -1.32327558]

[Text(1, 0, 'sepal length (cm)'),
 Text(2, 0, 'sepal width (cm)'),
 Text(3, 0, 'petal length (cm)'),
 Text(4, 0, 'petal width (cm)')]
```



Experiment 2: Linear Regression - Housing Price Prediction

Aim: To perform linear regression for housing-price prediction and observe the effects of outliers and feature scaling.

Code:

Dataset used: *California Housing dataset (sklearn.datasets.fetch_california_housing)*

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score

h = fetch_california_housing()
X = pd.DataFrame(h.data, columns=h.feature_names)
y = pd.Series(h.target, name="MedHouseValue")
print(X.head())
print("\nFeature names:", h.feature_names)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=.2, random_state=42)

raw = LinearRegression().fit(X_train, y_train)
pred = raw.predict(X_test)
print("\n--- Without Feature Scaling ---")
print("RMSE:", np.sqrt(mean_squared_error(y_test, pred)))
print("R2 Score:", r2_score(y_test, pred))

q1, q3 = X_train["AveRooms"].quantile([.25, .75])
iqr = q3 - q1
mask = (X_train["AveRooms"] < q1 - 1.5*iqr) | (X_train["AveRooms"] > q3 + 1.5*iqr)
print(f"\nNumber of outliers detected in AveRooms: {mask.sum()}")
Xc, yc = X_train[~mask], y_train[~mask]
clean = LinearRegression().fit(Xc, yc)
pred = clean.predict(X_test)
print("\n--- After Removing Outliers ---")
print("RMSE:", np.sqrt(mean_squared_error(y_test, pred)))
print("R2 Score:", r2_score(y_test, pred))

scaler = StandardScaler()
Xtr = scaler.fit_transform(X_train); Xte = scaler.transform(X_test)
scaled = LinearRegression().fit(Xtr, y_train)
pred = scaled.predict(Xte)
print("\n--- With Feature Scaling ---")
print("RMSE:", np.sqrt(mean_squared_error(y_test, pred)))
print("R2 Score:", r2_score(y_test, pred))

plt.figure(figsize=(6, 6))
plt.scatter(y_test, pred, alpha=.3)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], "r--")
plt.xlabel("Actual Price"); plt.ylabel("Predicted Price")
plt.title("Linear Regression: Actual vs Predicted")
```

Output:

```
MedInc HouseAge AveRooms AveBedrms Population AveOccup Latitude \
0 8.3252 41.0 6.984127 1.023810 322.0 2.555556 37.88
1 8.3014 21.0 6.238137 0.971880 2401.0 2.109842 37.86
2 7.2574 52.0 8.288136 1.073446 496.0 2.802260 37.85
3 5.6431 52.0 5.817352 1.073059 558.0 2.547945 37.85
4 3.8462 52.0 6.281853 1.081081 565.0 2.181467 37.85

Longitude
0 -122.23
1 -122.22
2 -122.24
3 -122.25
4 -122.25

Feature names: ['MedInc', 'HouseAge', 'AveRooms', 'AveBedrms', 'Population', 'AveOccup', 'Latitude', 'Longitude']

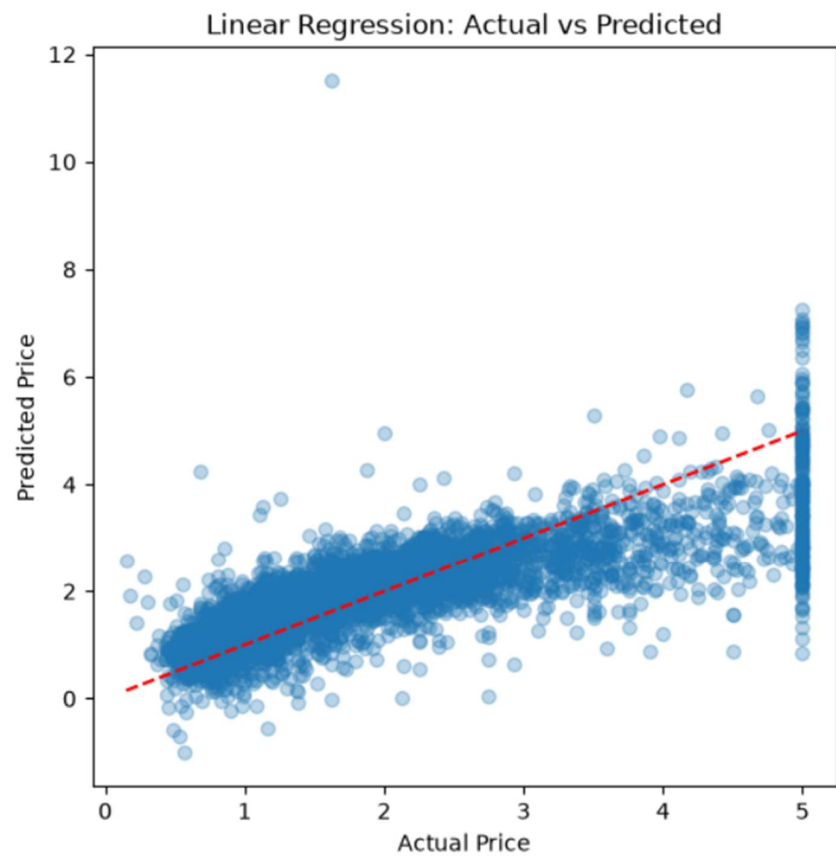
--- Without Feature Scaling ---
RMSE: 0.7455813830127763
R2 Score: 0.575787706032451

Number of outliers detected in AveRooms: 410

--- After Removing Outliers ---
RMSE: 0.8663223054890047
R2 Score: 0.42726709794427764

--- With Feature Scaling ---
RMSE: 0.7455813830127761
R2 Score: 0.5757877060324511
```

[2]: Text(0.5, 1.0, 'Linear Regression: Actual vs Predicted')



Experiment 3: Logistic Regression with Regularization

Aim: To study logistic regression under different regularization strengths and compare L1 and L2 penalties.

Code:

Dataset used: *Breast Cancer Wisconsin dataset* (`sklearn.datasets.load_breast_cancer`)

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix

d = load_breast_cancer()
X = pd.DataFrame(d.data, columns=d.feature_names)
y = pd.Series(d.target, name="target")
print("Dataset shape:", X.shape)
print("Classes:", d.target_names)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=.2, random_state=42,
stratify=y)

s = StandardScaler()
Xtr = s.fit_transform(X_train); Xte = s.transform(X_test)
Cs = [.001, .01, .1, 1, 10, 100]
results = []
for C in Cs:
    m = LogisticRegression(C=C, penalty="l2", solver="liblinear", max_iter=5000)
    m.fit(Xtr, y_train)
    a, b = m.score(Xtr, y_train), m.score(Xte, y_test)
    results.append((C, a, b))
    print(f'C={C:<8} Train Acc={a:.4f} Test Acc={b:.4f}')

m1 = LogisticRegression(C=1, penalty="l1", solver="liblinear", max_iter=5000).fit(Xtr, y_train)
m2 = LogisticRegression(C=1, penalty="l2", solver="liblinear", max_iter=5000).fit(Xtr, y_train)
print("\nL1 penalty test accuracy:", m1.score(Xte, y_test))
print("\nL2 penalty test accuracy:", m2.score(Xte, y_test))
print("Non-zero coefficients (L1):", np.sum(m1.coef_ != 0))
print("Non-zero coefficients (L2):", np.sum(m2.coef_ != 0))

best = LogisticRegression(C=1, penalty="l2", solver="liblinear", max_iter=5000).fit(Xtr,
y_train)
yp = best.predict(Xte)
print("\nClassification Report:\n", classification_report(y_test, yp,
target_names=d.target_names))
print("Confusion Matrix:\n", confusion_matrix(y_test, yp))

Cs, train_accs, test_accs = zip(*results)
plt.figure(figsize=(7, 5))
plt.plot(Cs, train_accs, marker="o", label="Train Accuracy")
plt.plot(Cs, test_accs, marker="s", label="Test Accuracy")
plt.xscale("log"); plt.xlabel("Regularization Parameter C (log scale)")
plt.ylabel("Accuracy"); plt.title("Effect of Regularization on Logistic Regression")
```

Output:

Dataset shape: (569, 30)
Classes: ['malignant' 'benign']
C=0.001 Train Acc=0.9538 Test Acc=0.9386
C=0.01 Train Acc=0.9736 Test Acc=0.9561
C=0.1 Train Acc=0.9868 Test Acc=0.9825
C=1 Train Acc=0.9890 Test Acc=0.9825
C=10 Train Acc=0.9912 Test Acc=0.9649
C=100 Train Acc=0.9934 Test Acc=0.9474

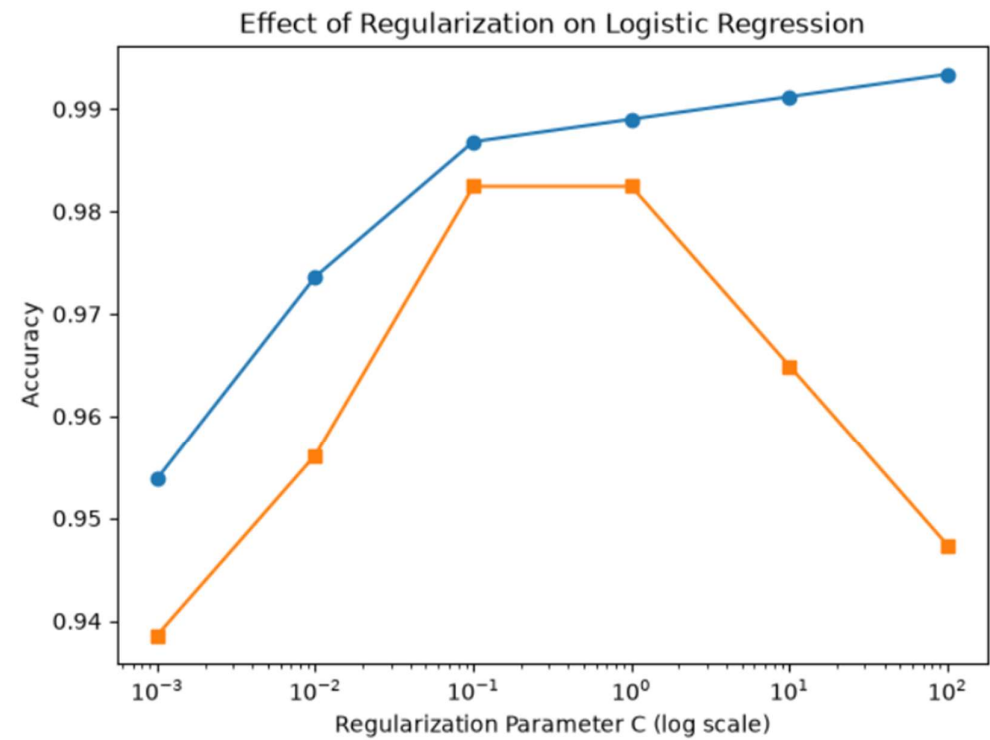
L1 penalty test accuracy: 0.9912280701754386
L2 penalty test accuracy: 0.9824561403508771
Non-zero coefficients (L1): 16
Non-zero coefficients (L2): 30

Classification Report:

	precision	recall	f1-score	support
malignant	0.98	0.98	0.98	42
benign	0.99	0.99	0.99	72
accuracy			0.98	114
macro avg	0.98	0.98	0.98	114
weighted avg	0.98	0.98	0.98	114

Confusion Matrix:
[[41 1]
[1 71]]

Text(0.5, 1.0, 'Effect of Regularization on Logistic Regression')



Experiment 4: K-Nearest Neighbors vs Decision Tree

Aim: To compare K-Nearest Neighbors and a Decision Tree while studying the effect of different K values on KNN accuracy.

Code:

Dataset used: *Wine dataset (sklearn.datasets.load_wine)*

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report

d = load_wine()
X = pd.DataFrame(d.data, columns=d.feature_names)
y = pd.Series(d.target, name="target")
print("Dataset shape:", X.shape)
print("Classes:", d.target_names)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=.25, random_state=42,
stratify=y)

s = StandardScaler()
Xtr = s.fit_transform(X_train); Xte = s.transform(X_test)
ks, accs = range(1, 21), []
for k in ks:
    m = KNeighborsClassifier(n_neighbors=k).fit(Xtr, y_train)
    a = m.score(Xte, y_test); accs.append(a)
    print(f"K={k:<3} Test Accuracy={a:.4f}")
best_k = ks[np.argmax(accs)]
print(f"\nBest K: {best_k} with accuracy {max(accs):.4f}")
knn = KNeighborsClassifier(n_neighbors=best_k).fit(Xtr, y_train)
kp = knn.predict(Xte)
print("\nKNN Classification Report:\n", classification_report(y_test, kp,
target_names=d.target_names))

tree = DecisionTreeClassifier(max_depth=4, random_state=42).fit(X_train, y_train)
tp = tree.predict(X_test); tree_acc = accuracy_score(y_test, tp)
print(f"\nDecision Tree Test Accuracy: {tree_acc:.4f}")
print("Decision Tree Classification Report:\n", classification_report(y_test, tp,
target_names=d.target_names))
print(f"\nComparison -> KNN (K={best_k}): {max(accs):.4f} | Decision Tree: {tree_acc:.4f}")

plt.figure(figsize=(7, 5))
plt.plot(ks, accs, marker="o", label="KNN Accuracy")
plt.axhline(y=tree_acc, color="r", linestyle="--", label="Decision Tree Accuracy")
plt.xlabel("Number of Neighbors (K)"); plt.ylabel("Test Accuracy")
plt.title("Effect of K on KNN Accuracy vs Decision Tree")
```

Output:

```
Dataset shape: (178, 13)
Classes: ['class_0' 'class_1' 'class_2']
K=1  Test Accuracy=0.9556
K=2  Test Accuracy=0.9333
K=3  Test Accuracy=0.9556
K=4  Test Accuracy=0.9333
K=5  Test Accuracy=0.9333
K=6  Test Accuracy=0.9333
K=7  Test Accuracy=0.9778
K=8  Test Accuracy=0.9556
K=9  Test Accuracy=0.9778
K=10 Test Accuracy=0.9778
K=11 Test Accuracy=0.9778
K=12 Test Accuracy=0.9778
K=13 Test Accuracy=0.9778
K=14 Test Accuracy=0.9778
K=15 Test Accuracy=1.0000
K=16 Test Accuracy=0.9778
K=17 Test Accuracy=1.0000
K=18 Test Accuracy=1.0000
K=19 Test Accuracy=1.0000
K=20 Test Accuracy=1.0000
```

Best K: 15 with accuracy 1.0000

Best K: 15 with accuracy 1.0000

KNN Classification Report:

	precision	recall	f1-score	support
class_0	1.00	1.00	1.00	15
class_1	1.00	1.00	1.00	18
class_2	1.00	1.00	1.00	12
accuracy			1.00	45
macro avg	1.00	1.00	1.00	45
weighted avg	1.00	1.00	1.00	45

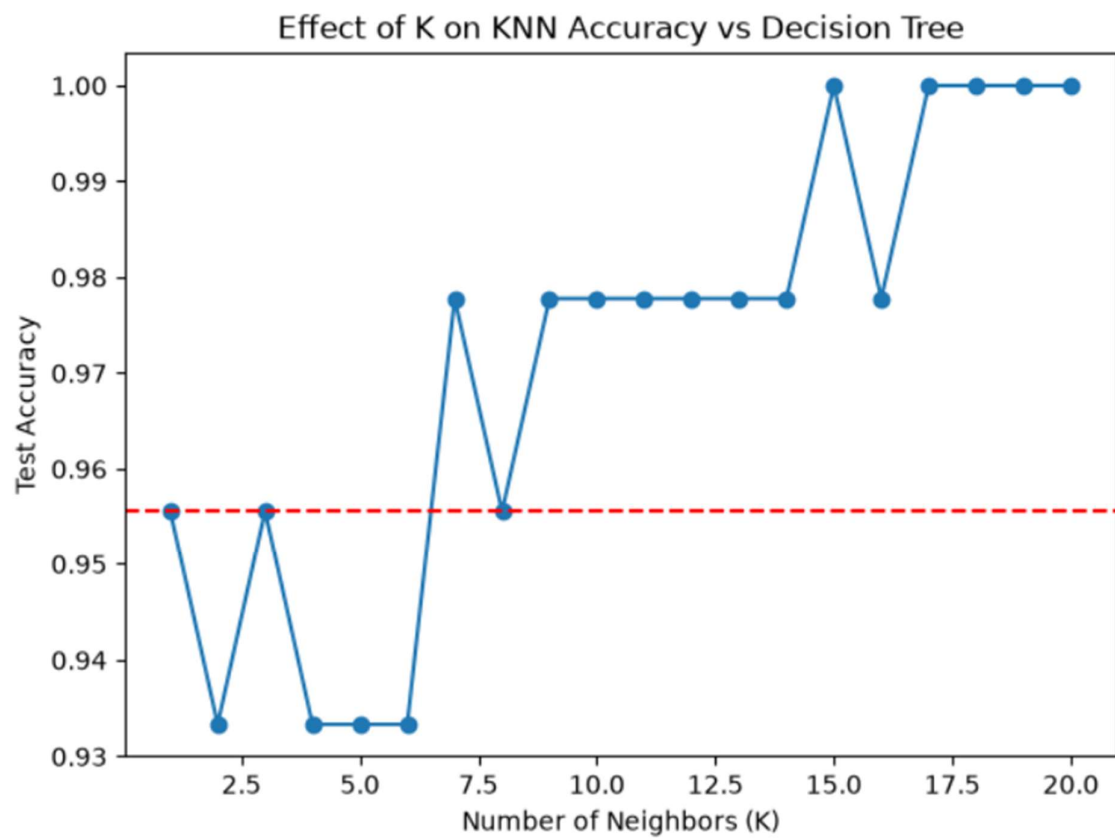
Decision Tree Test Accuracy: 0.9556

Decision Tree Classification Report:

	precision	recall	f1-score	support
class_0	1.00	0.93	0.97	15
class_1	0.90	1.00	0.95	18
class_2	1.00	0.92	0.96	12
accuracy			0.96	45
macro avg	0.97	0.95	0.96	45
weighted avg	0.96	0.96	0.96	45

Comparison -> KNN (K=15): 1.0000 | Decision Tree: 0.9556

Text(0.5, 1.0, 'Effect of K on KNN Accuracy vs Decision Tree')



Experiment 5: Support Vector Machine with Different Kernels

Aim: To compare SVM models using linear, polynomial, RBF, and sigmoid kernels and visualize their decision boundaries.

Code:

Dataset used: Iris dataset (*sklearn.datasets.load_iris*), 2 features used for 2D visualization

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

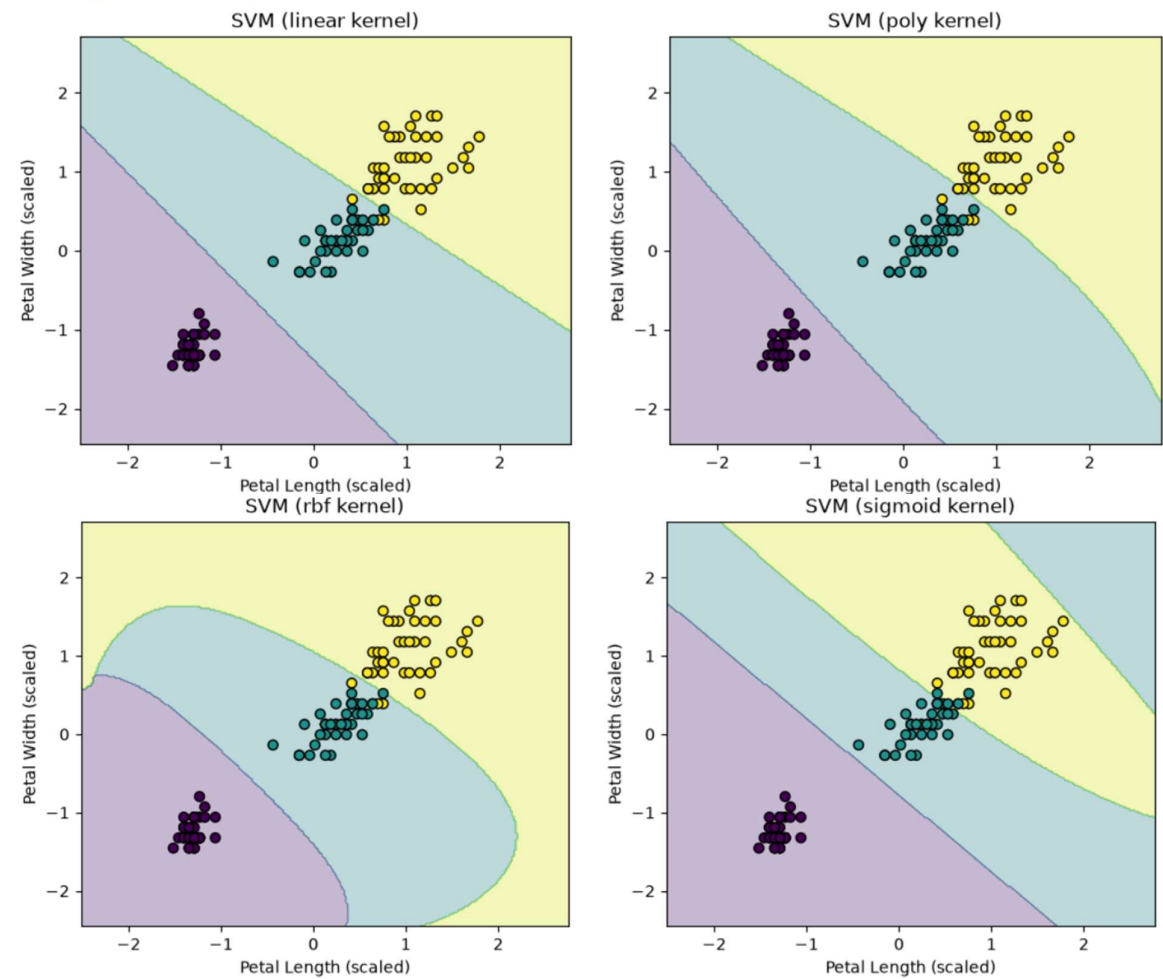
iris = load_iris()
X, y = iris.data[:, [2, 3]], iris.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=.2, random_state=42,
stratify=y)
s = StandardScaler()
Xtr = s.fit_transform(X_train); Xte = s.transform(X_test)
kernels, models = ["linear", "poly", "rbf", "sigmoid"], {}
for k in kernels:
    m = SVC(kernel=k, C=1.0, gamma="scale", degree=3, random_state=42).fit(Xtr, y_train)
    print(f"Kernel={k:<8} Test Accuracy={accuracy_score(y_test, m.predict(Xte)):.4f}")
    models[k] = m

def boundary(model, X, y, ax, title):
    h = .02
    x0, x1 = X[:, 0].min()-1, X[:, 0].max()+1
    y0, y1 = X[:, 1].min()-1, X[:, 1].max()+1
    xx, yy = np.meshgrid(np.arange(x0, x1, h), np.arange(y0, y1, h))
    Z = model.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)
    ax.contourf(xx, yy, Z, alpha=.3, cmap="viridis")
    ax.scatter(X[:, 0], X[:, 1], c=y, edgecolors="k", cmap="viridis")
    ax.set_title(title); ax.set_xlabel("Petal Length (scaled)"); ax.set_ylabel("Petal Width
(scaled)")

fig, axes = plt.subplots(2, 2, figsize=(12, 10))
for ax, k in zip(axes.ravel(), kernels):
    boundary(models[k], Xtr, y_train, ax, f"SVM ({k} kernel)")
```

Output:

Kernel=linear Test Accuracy=0.9333
Kernel=poly Test Accuracy=0.9333
Kernel=rbf Test Accuracy=0.9333
Kernel=sigmoid Test Accuracy=0.9333



Experiment 6: Perceptron & Neural Network with Backpropagation

Aim: To implement a perceptron with manual backpropagation and study a neural network with different activation functions on the Digits dataset.

Code:

Dataset used: Part A: toy AND-gate data (manual). Part B: Digits dataset
(`sklearn.datasets.load_digits`)

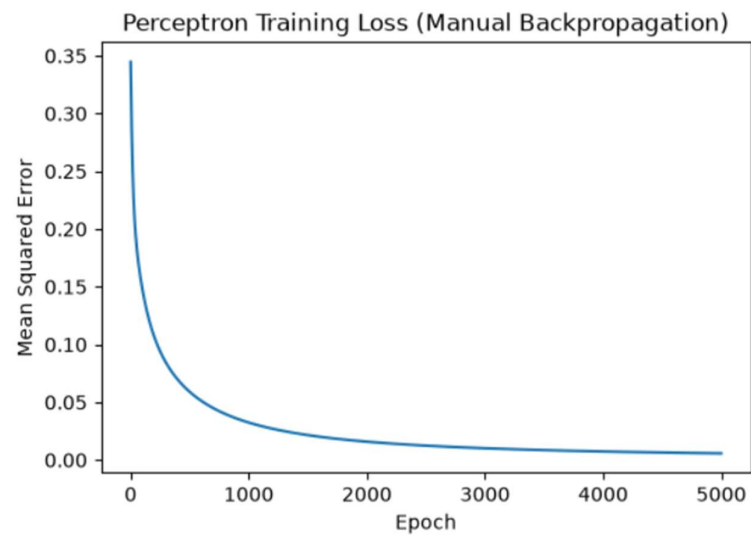
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

# Part A: AND gate perceptron
X_gate = np.array([[0,0],[0,1],[1,0],[1,1]])
y_gate = np.array([[0],[0],[0],[1]])
def sigmoid(x): return 1/(1+np.exp(-x))
def sigmoid_derivative(x): return x*(1-x)
np.random.seed(42)
w = np.random.uniform(-1, 1, (2, 1)); b = np.random.uniform(-1, 1, (1,))
lr, epochs, losses = .1, 5000, []
for _ in range(epochs):
    out = sigmoid(np.dot(X_gate, w) + b)
    err = y_gate - out; losses.append(np.mean(err**2))
    grad = err * sigmoid_derivative(out)
    w += lr*np.dot(X_gate.T, grad); b += lr*np.sum(grad)
print("Predictions after training perceptron on AND gate:")
print(np.round(out, 3))
print("Final weights:", w.ravel(), "Bias:", b)
plt.figure(figsize=(6,4)); plt.plot(losses)
plt.xlabel("Epoch"); plt.ylabel("Mean Squared Error")
plt.title("Perceptron Training Loss (Manual Backpropagation)")
plt.savefig("perceptron_loss.png"); plt.show()

# Part B: Digits neural network
digits = load_digits()
X, y = digits.data, digits.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=.2, random_state=42,
stratify=y)
s = StandardScaler()
X_train = s.fit_transform(X_train); X_test = s.transform(X_test)
acts, histories = ["relu", "sigmoid", "tanh"], {}
for act in acts:
    model = keras.Sequential([layers.Input(shape=(64,)), layers.Dense(32, activation=act),
layers.Dense(16, activation=act), layers.Dense(10,
activation="softmax")])
    model.compile(optimizer="adam", loss="sparse_categorical_crossentropy",
metrics=["accuracy"])
    hist = model.fit(X_train, y_train, epochs=30, batch_size=16, validation_split=.2, verbose=0)
    loss, acc = model.evaluate(X_test, y_test, verbose=0); histories[act] = hist
    print(f"Activation={act:<8} Test Accuracy={acc:.4f} Test Loss={loss:.4f}")
plt.figure(figsize=(7,5))
for act in acts: plt.plot(histories[act].history["val_accuracy"], label=f"{act} (val)")
plt.xlabel("Epoch"); plt.ylabel("Validation Accuracy")
plt.title("Effect of Activation Function on Neural Network Performance")
plt.legend(); plt.savefig("activation_comparison.png"); plt.show()
```

Output:

Predictions after training perceptron on AND gate:
[[0.001]
[0.082]
[0.082]
[0.902]]
Final weights: [4.64400325 4.64400422] Bias: [-7.06339674]



WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.
Activation=relu Test Accuracy=0.9722 Test Loss=0.1067
Activation=sigmoid Test Accuracy=0.9444 Test Loss=0.2582
Activation=tanh Test Accuracy=0.9583 Test Loss=0.1540

